

a reflection with determinant -1 . Thus, to get determinant 1, we do nothing to $\mathbf{Q}$ if n is odd; if n is even, we simply interchange any pair of rows in $\mathbf{Q}$, giving the final answer.

For large n the work of doing the decomposition scales as $O(n^3)$, which can be burdensome. For faster, but more complicated, methods, see [2,3].

Faster special methods are available for two and three dimensions. A random two-dimensional rotation matrix has components that are the sine and cosine of a random angle θ in $[0, 2\pi]$,

$$\begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \quad (21.5.16)$$

We get the components without trigonometric function calls by using (21.5.12) to find a random point on the unit circle and then taking $\cos \theta = x_0$ and $\sin \theta = x_1$.

In the case of three dimensions, a fast method is to use equation (21.5.14) to generate a random point on the sphere in four dimensions, and then to construct the 3×3 orthogonal matrix,

$$\begin{bmatrix} 1 - 2(x_1^2 + x_2^2) & 2(x_0x_1 - x_3x_2) & 2(x_0x_2 + x_3x_1) \\ 2(x_0x_1 + x_3x_2) & 1 - 2(x_0^2 + x_2^2) & 2(x_1x_2 - x_3x_0) \\ 2(x_0x_2 - x_3x_1) & 2(x_1x_2 + x_3x_0) & 1 - 2(x_0^2 + x_1^2) \end{bmatrix} \quad (21.5.17)$$

which will be uniformly random among all rotations [4,5].

CITED REFERENCES AND FURTHER READING:

- Marsaglia, G. 1972, "Choosing a Point from the Surface of a Sphere," *Annals of Mathematical Statistics*, vol. 43, pp. 645–646.[1]
- Genz, A. 2000, "Methods for Generating Random Orthogonal Matrices," in *Monte Carlo and Quasi-Monte Carlo Methods*, Proceedings of the Third International Conference on Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing (MCQMC98) (Berlin: Springer).[2]
- Anderson, T.W., Olkin, I., and Underhill, L.G. 1987, "Generation of Random Orthogonal Matrices," *SIAM Journal on Scientific and Statistical Computing*, vol. 8, pp. 625–629.[3]
- Shoemake, K. 1985, "Animating Rotation with Quaternion Curves," *Computer Graphics*, Proceedings of SIGGRAPH 1985, vol. 19, pp. 245–254.[4]
- Shoemake, K. 1992, "Uniform Random Rotations," in *Graphics Gems III*, Kirk, D., ed. (Cambridge, MA: Academic Press), pp. 124–132.[5]

21.6 Triangulation and Delaunay Triangulation

We can informally define a *triangulation* of a set of N points in the plane as follows: Connect the given points by straight-line segments as many times as you can without any two segments crossing. When you can't connect any more, you have a triangulation. Obviously there are many triangulations of a given set of points. Figure 21.6.1 shows three triangulations of the same set of 50 points. Two are "random," where the informal definition was pretty much followed literally. The third one is a very special triangulation, called a Delaunay triangulation. In a sense that we will make more precise below, it is the triangulation whose triangles best avoid small angles and long sides.

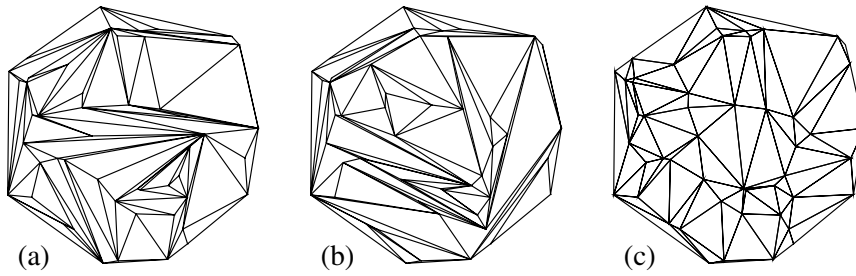


Figure 21.6.1. Three triangulations of the same 50 random points: (a) and (b) are “bad” (random) triangulations, while (c) is a “good” (Delaunay) triangulation. The number of lines and triangles is the same in each case.

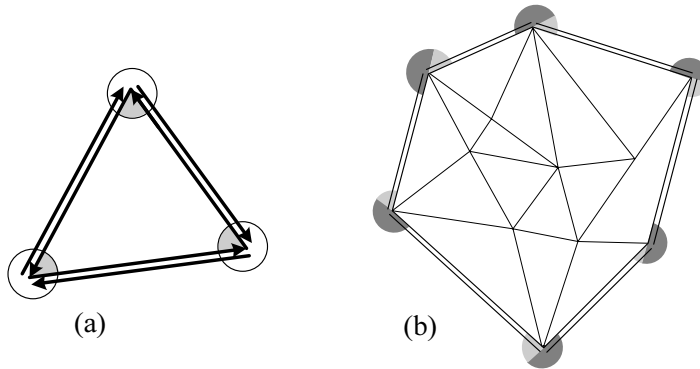


Figure 21.6.2. How to count lines and triangles in a triangulation. (a) Each triangle “uses up” $1/2$ of a point, and $3/2$ lines. (b) The n points in the convex hull “use up” $n/2 + 1$ points and $n/2$ lines.

All triangulations of a given set of points have the same outer boundary, called the *convex hull* of the point set. This should be evident, again informally, from the definition of triangulation: A line segment (edge) on the outer convex boundary can’t interfere with any other actual or potential interior edges, so it will always be added before the stopping rule is reached. The number of points n (and also edges) in the convex hull is at least three, and can be as large as N , e.g., if the points all lie on a circle. (Here and below, we will ignore degenerate cases like “all points lie on a line.”)

It is surprising, perhaps, that all triangulations of a given point set have the same number of lines (L) and triangles (T), given explicitly by the relations

$$\begin{aligned} L &= 3N - n - 3 \\ T &= 2N - n - 2 \end{aligned} \tag{21.6.1}$$

The proof, known to Gauss, is very easy if you consult Figure 21.6.2. Since the interior angles of a triangle sum to π radians, each triangle “uses up” half a point’s worth of angle. It is useful to think about each line as being two half-lines, representing the two possible directions of traversal in clockwise triangles. Then, each triangle uses up three half-lines. We must separately account for the vertices on the convex hull, as follows: Each such point uses up π radians on its own (dark-shaded angles in the figure), plus (sum of the light-shaded exterior angles) 2π additional radians in going

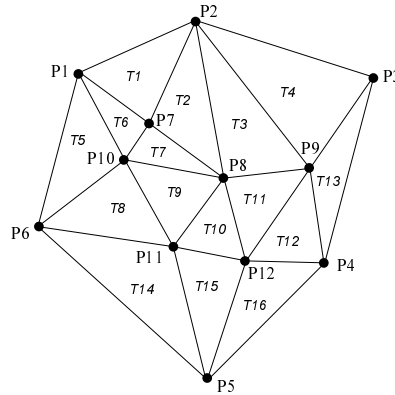


Figure 21.6.3. Example of a triangulation with $N = 12$, $n = 6$, $T = 16$, and $L = 27$, values that satisfy equation (21.6.1).

around the convex hull. These considerations give the relations

$$\begin{aligned} 2\pi N &= \pi T + \pi n + 2\pi && \text{(account for radians)} \\ 2L &= 3T + n && \text{(account for half-lines)} \end{aligned} \quad (21.6.2)$$

which can be rearranged to give equation (21.6.1). Figure 21.6.3 shows the triangulation of Figure 21.6.2(b) with its points and triangles enumerated.

21.6.1 Delaunay Triangulation

Boris Nikolaevich Delone (1890–1980), a Russian mathematician also celebrated as a rock climber, first published the ideas behind Delaunay triangulation in 1934. Since his paper was written in French, his name was transliterated so as to be pronounced (approximately) correctly by French speakers. A better English pronunciation might be “dyeh-LOAN-yeh.”

Delaunay triangulations have a number of remarkable properties and can be defined in various abstract ways. However, we’ll take as the definition one very concrete property, shown in Figure 21.6.4. Consider all triangulations in which four points, A, B, C, D , are the vertices of two back-to-back triangles. Then, one can get a different triangulation by deleting the common edge (BD in the figure) and replacing it by the other diagonal of the quadrilateral (AC in the figure). A Delaunay triangulation is defined as one that always chooses the diagonal that gives the *largest minimum angle* for the six interior angles in the two triangles. The edge shown as BD is thus *illegal* for a Delaunay triangulation, while AC is termed *legal*. Changing a triangulation from an illegal edge to a legal one is called an *edge flip*. When any two triangles have a common edge, exactly one configuration, unchanged or edge-flipped, is legal (unless all four points lie on a circle, in which case both are legal).

This “largest minimum angle” property is geometrically equivalent to other statements about the points A, B, C, D . One such statement is that the circumcircle of an illegal triangle, like ABD or BCD in part (a) of the figure, always contains another point, C or A , respectively. For a legal triangle, as in part (b) of the figure, this is never the case. One can use this as a starting point to prove the theorem:

- The circumcircle of any triangle in a Delaunay triangulation contains no other vertices.

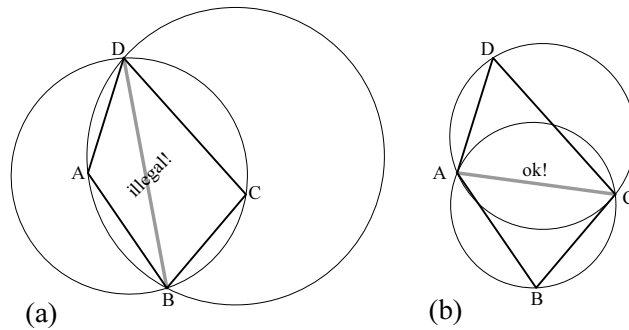


Figure 21.6.4. A Delaunay triangulation can be defined as one in which back-to-back triangles always have a larger minimum angle than they would have if their common edge were flipped to the other diagonal. Equivalently, any triangle's circumcircle does not contain another vertex. A so-called *edge flip* converts (a) to (b) in the figure.

Although the largest minimum angle property was defined locally, for one quadrilateral at a time, it can be shown to imply a remarkable global theorem:

- Among all triangulations of a set of points, a Delaunay triangulation has the largest minimum angles, as defined by sorting all angles from smallest to largest and comparing lexicographically to the angles of any other triangulation.

Comparing lexicographically means: First compare the smallest angle; if there is a winner, stop. If there is a tie on the smallest angle, compare the second-smallest angle. And so on.

Another theorem is

- Two vertices are connected by a Delaunay edge if and only if there is some circle that contains them and contains no other vertices.

If the points in a set have generic positions, meaning that no three are collinear and no four lie on the same circle, then the Delaunay triangulation exists and is unique; any method for constructing it will give the identical set of triangles.

You might wonder whether a Delaunay triangulation is also a *minimum weight* triangulation, defined as the triangulation with the smallest total of edge lengths. The answer is, in general, no. While minimum weight triangulations might be useful in applications, it is not even known whether they can be constructed in less than time that grows exponentially with N . Delaunay construction, on the other hand, is fast, $O(N \log N)$. So, in practice, Delaunay is what we've got!

So, how do we construct a Delaunay triangulation? Conceptually, we can start with *any* triangulation and then eliminate illegal edges, by edge flips, as long as possible. This must terminate in a Delaunay triangulation after a finite number of flips because (i) each flip changes and increases the lexicographic order in the list of angles, and (ii) there are only a finite number of possible triangulations. Although, as stated, this is not an efficient algorithm, it can be readily be turned into one, the so-called *randomized incremental algorithm* [1].

This algorithm, which we now implement, is “incremental” in that it adds points to the triangulation one at a time, maintaining a Delaunay triangulation at each stage. It is “randomized” in that the points are added in a random order. It turns out that the randomization (almost) guarantees $O(N \log N)$ expected time for the algorithm. (Without randomization, one could encounter pathological cases with

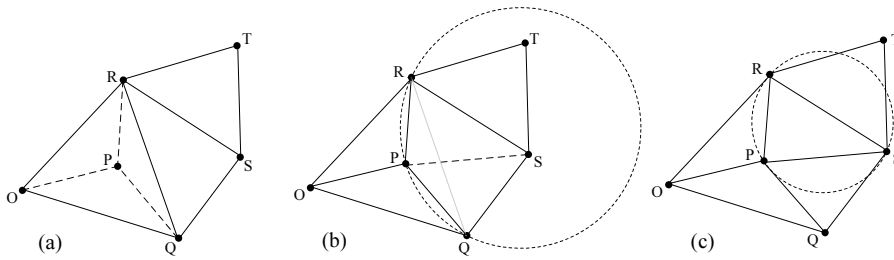


Figure 21.6.5. Steps in inserting a new point into a Delaunay triangulation. (a) Connect new point P to vertices of enclosing triangle. (b) Check enclosing triangle for illegal edges (here, replace QR by PS). (c) Recursively check any new triangles created that have P as a vertex. (Here, RS is legal, so we terminate.)

$O(N^2)$ running time.)

Figure 21.6.5 shows the procedure for adding a new point P that lies within an existing triangle. First, connect it to the vertices of the enclosing triangle. This creates three new triangles. (We exclude the special case where P is exactly on an existing line. More on this below.) Next, check whether the edges opposite P in the three new triangles are legal or illegal. If they are illegal, do edge flips. Each edge flip creates two new triangles with P as a vertex, and (therefore) with two edges opposite P that now also need to be checked for legality. So the process is recursive, but it never wanders away from P . That is the key point: The only edges that can be made illegal by inserting point a point P are edges opposite P in triangles that include P . The proof that the algorithm is $O(N \log N)$ uses this fact, and then bounds the average number of triangles of which P can be a vertex by relations like those of equation (21.6.1). (For details in the proof, see [2].)

Since, up to now, we only know how to add a point P that falls inside the triangulation, how can we get started? An easy way is to add three “fictitious” points to the set of points, forming a very large starting triangle that will enclose all the “real” points subsequently added. Then, at the very end, the fictitious points and all edges connecting to them are removed. Strictly speaking, the fictitious points must actually be treated as if at infinity (thus requiring special logic in the code whenever they are referenced). If their distance is merely finite, the constructed triangulation may be “not-quite” Delaunay. For example, its outer boundary (convex hull) could in unusual cases be left slightly concave, with small negative angles on the order of the diameter of the “real” point set divided by the distance to the “fictitious” points.

That’s enough general information. We next get into the details.

21.6.2 Implementation Details

Since most readers skip a section with this title, this is a good place for us to confess a couple of dirty tricks in our Delaunay implementation, whose purpose is to keep the code and its explication to manageable length. If you need a bullet-proof Delaunay code, with no such tricks, a Web search will turn up several that are freely available. Our code is short and fast, and fine for its intended purpose; but it is approximate in two respects: First, we don’t take the initial bounding triangle off to infinity (as we sanctimoniously advised, above). Instead, it lies at a distance of about `bigscale` (an adjustable parameter, default value 1000), measured in units of the bounding-box size of the set of points. Second, we don’t provide for the special case, mentioned above, where the point being added falls on an existing edge (or does so within roundoff tolerance). For generic point locations, this should “never” happen; but in real

life it “always” happens, because users love to try test examples with points in regular patterns! When we detect this problem, we randomly alter the location of the offending point by a small fraction `fuzz` (another adjustable parameter, default 10^{-6}) of the bounding box dimensions.

A very important implementation detail, not yet discussed, is how to find in which existing triangle a new point lies. Conceptually, we might throw the triangles into a QO tree (§21.8), but this would not yield the desired $O(N \log N)$ behavior for our algorithm. A better solution, well established in the literature, is to maintain a tree structure of the descendants of any given triangle that ever existed in the construction. That is, starting with the huge “root” triangle, whenever a triangle is subdivided into three new triangles, we set pointers to its three daughters. And, when two triangles are lost in an edge flip, and two new triangles are created, we make the new triangles daughters of *both* of the lost triangles (even though each lost triangle contains only a part of each new one). In this scheme a triangle has two or three daughters at most, so we can easily reserve space for the pointers explicitly (i.e., no expandable linked lists needed).

With this structure, it is very fast to locate a point within the existing triangulation: Start at the root triangle, and recursively pick whichever daughter contains the point. When you reach a terminal node in the tree, you will have found a triangle in the current triangulation that contains the point. We thus need a structure for a “triangle element” or `Triel`:

`delanay.h`

```
struct Triel {
    Structure for an element in a descendency tree of triangles, each having at most three daughters.
    Point<2> *pts;           Pointer to the array of the points.
    Int p[3];               The triangle's three vertices, always in CCW order.
    Int d[3];               Pointers for up to three daughters.
    Int stat;               Nonzero if this element is "live."
    void setme(Int a, Int b, Int c, Point<2> *ptss) {
        Set the data in a Triel.
        pts = ptss;
        p[0] = a; p[1] = b; p[2] = c;
        d[0] = d[1] = d[2] = -1;           The values -1 mean no daughters.
        stat = 1;                         Create as "live."
    }
    Int contains(Point<2> point) {
        Return 1 if point is in the triangle, 0 if on boundary, -1 if outside. (CCW triangle is
        assumed.)
        Doub d;
        Int i,j,ztest = 0;
        for (i=0; i<3; i++) {
            j = (i+1) %3;
            d = (pts[p[j]].x[0]-pts[p[i]].x[0])*(point.x[1]-pts[p[i]].x[1]) -
                (pts[p[j]].x[1]-pts[p[i]].x[1])*(point.x[0]-pts[p[i]].x[0]);
            if (d < 0.0) return -1;
            if (d == 0.0) ztest = 1;
        }
        return (ztest? 0 : 1);
    }
};
```

We create an big enough array of `Triels` at the start, and use integers to point to array elements. We’ve omitted any explicit constructor or assignment operators in `Triel`, since they are not needed for our use here. Be sure to add them if you use `Triel` in any other way.

We will need a way to do two other fast lookups: (1) Given a point and opposite edge in a triangle, find the fourth point in the quadrilateral, that is, the point (if any) on the other side of the given edge. (2) Given three points, find the index of their triangle (if it exists) in an array of `Triel` elements. Our strategy is to use hash memories (respectively called `linehash` and `trihash`) for these two functions. In particular, whenever we create a triangle (always CCW) with vertices A, B, C , we store an index pointing to each point under a specially constructed key,

$$\text{linehash}(h(B) - h(C)) \leftarrow A \quad (\text{et cyc.}) \quad (21.6.3)$$

where the function h is a 64-bit hash function, and “et cyc.” means do the same thing for the

other two cyclic permutations of A, B, C . The trick here is that, if we ever want to find the point on the other side of edge BC , we just look for a key $h(C) - h(B)$ (“negative” of the key in equation 21.6.3) in the hash table. The similar trick for storing and retrieving `Triels` is that, when we create a `Triel` at location j in the storage array, we set

$$\text{trihash}(h(A) \wedge h(B) \wedge h(C)) \leftarrow j \quad (21.6.4)$$

where $\wedge$ is the XOR operation. Since this key is symmetric in A, B, C , we can find a triangle knowing its vertices in any order.

Since we are computing hash keys “by hand,” we can signal the two hash memories to use a null (therefore fast) hash of their own. That, and a utility for determining whether a point is inside the circumcircle of three other points, are the following two code fragments:

```
Doub incircle(Point<2> d, Point<2> a, Point<2> b, Point<2> c) {           delaunay.h
Return positive, zero, or negative value if point d is respectively inside, on, or outside the circle
through points a, b, and c.
    Circle cc = circumcircle(a,b,c);           Routine defined in §21.3.
    Doub radd = SQR(d.x[0]-cc.center.x[0]) + SQR(d.x[1]-cc.center.x[1]);
    return (SQR(cc.radius) - radd);
}

struct Nullhash {
Null hash function. Use a key (assumed to be already hashed) as its own hash.
    Nullhash(Int nn) {}
    inline Ullong fn(const void *key) const { return *((Ullong *)key); }
};
```

These are all the preliminaries we need before declaring the Delaunay structure.

```
struct Delaunay {                                                       delaunay.h
Structure for constructing a Delaunay triangulation from a given set of points.
    Int npts,ntri,ntree,nreemax,opt;           Number of points, triangles, elements in the
    Doub delx,dely;                           Triel list, and maximum of same. Size
    vector< Point<2> > pts;                    of the bounding box.
    vector<Triel> thelist;                    The list of Triel elements.
    Hash<Ullong,Int,Nullhash> *linehash;      Create the hash memories with null hash
    Hash<Ullong,Int,Nullhash> *trihash;       function.
    Int *perm;
    Delaunay(vector<Point<2> > &pvec, Int options = 0);
Construct the Delaunay triangulation from a vector of points. The variable options is used
by some applications.
    Ranhash hashfn;                           The raw hash function.
    Doub interpolate(const Point<2> &p, const vector<Doub> &fnvals,
        Doub defaultval=0.0);
The next four functions are explained in detail below.
    void insertapoint(Int r);
    Int whichcontainspt(const Point<2> &p, Int strict = 0);
    Int storetriangle(Int a, Int b, Int c);
    void erasetriangle(Int a, Int b, Int c, Int d0, Int d1, Int d2);
    static UInt jran;                          Random number counter.
    static const Doub fuzz, bigscale;
};
const Doub Delaunay::fuzz = 1.0e-6;           Adjust if you wish. See text.
const Doub Delaunay::bigscale = 1000.0;      Adjust if you wish. See text.
UInt Delaunay::jran = 14921620;
```

The variable `jran` is used in conjunction with the hash function as a convenient random number generator. The function `interpolate()` is for the application of interpolating a function on an irregular mesh, to be discussed in §21.7. Everything else should become clear as we proceed.

The action starts with the constructor. We compute a bounding box for the set of points, construct and store the “huge” root triangle enclosing the points, create a random permuta-

tion to be the order in which points are added, and then (for the real work) call the function `insertapoint()` for each point in turn. After that there is just some cleanup housekeeping.

```
delaunay.h Delaunay::Delaunay(vector< Point<2> > &pvec, Int options) :
    npts(pvec.size()), ntri(0), ntree(0), ntreemax(10*npts+1000),
    opt(options), pts(npts+3), thelist(ntreemax) {
Construct Delaunay triangulation from a vector of points pvec. If bit 0 in options is nonzero,
hash memories used in the construction are deleted. (Some applications may want to use them
and will set options to 1.)
    Int j;
    Doub xl,xh,yl,yh;
    linehash = new Hash<Ullong,Int,Nullhash>(6*npts+12,6*npts+12);
    trihash = new Hash<Ullong,Int,Nullhash>(2*npts+6,2*npts+6);
    perm = new Int[npts];           Permutation for randomizing point order.
    xl = xh = pvec[0].x[0];        Copy points to local store and calculate their
    yl = yh = pvec[0].x[1];        bounding box.
    for (j=0; j<npts; j++) {
        pts[j] = pvec[j];
        perm[j] = j;
        if (pvec[j].x[0] < xl) xl = pvec[j].x[0];
        if (pvec[j].x[0] > xh) xh = pvec[j].x[0];
        if (pvec[j].x[1] < yl) yl = pvec[j].x[1];
        if (pvec[j].x[1] > yh) yh = pvec[j].x[1];
    }
    delx = xh - xl;                Store bounding box dimensions, then construct
    dely = yh - yl;                the three fictitious points and store them.
    pts[npts] = Point<2>(0.5*(xl + xh), yh + bigscale*dely);
    pts[npts+1] = Point<2>(xl - 0.5*bigscale*delx, yl - 0.5*bigscale*dely);
    pts[npts+2] = Point<2>(xh + 0.5*bigscale*delx, yl - 0.5*bigscale*dely);
    storetriangle(npts,npts+1,npts+2);
    Create a random permutation:
    for (j=npts; j>0; j--) SWAP(perm[j-1],perm[hashfn.int64(jran++) % j]);
    for (j=0; j<npts; j++) insertapoint(perm[j]);    All the action is here!
    for (j=0; j<ntree; j++) {
        Delete the huge root triangle and all of its con-
        if (thelist[j].stat > 0) {
            if (thelist[j].p[0] >= npts || thelist[j].p[1] >= npts ||
                thelist[j].p[2] >= npts) {
                thelist[j].stat = -1;
                ntri--;
            }
        }
    }
    if (!(opt & 1)) {                Clean up, unless option bit says not to.
        delete [] perm;
        delete trihash;
        delete linehash;
    }
}
```

The guts of the algorithm as previously described are in `insertapoint()`. We first locate the triangle that contains the new point. (Failure here can mean only that the point lies on an existing line, in which case we fuzz the point's location, as we confessed above, and try again.) We store three new triangles and delete the old one. Then, we locate and fix any illegal edges, doing the recursion by a simple last-in-first-out stack of pending edges to check.

```
delaunay.h void Delaunay::insertapoint(Int r) {
Add the point with index r incrementally to the Delaunay triangulation.
    Int i,j,k,l,s,tno,ntask,d0,d1,d2;
    Ullong key;
    Int tasks[50], taski[50], taskj[50];
    for (j=0; j<3; j++) {
        tno = whichcontainspt(pts[r],1);
        if (tno >= 0) break;
        Stacks (3 vertices) for legalizing edges.
        Find triangle containing point. Fuzz if it
        lies on an edge.
        The desired result: Point is OK.
    }
```



```

    pts[r].x[0] += fuzz * delx * (hashfn.doub(jran++)-0.5);
    pts[r].x[1] += fuzz * dely * (hashfn.doub(jran++)-0.5);
}
if (j == 3) throw("points degenerate even after fuzzing");
ntask = 0;
i = thelist[tno].p[0]; j = thelist[tno].p[1]; k = thelist[tno].p[2];
The following line is relevant only when the indicated bit in opt is set. This feature is used
by the convex hull application and causes any points already known to be interior to the
convex hull to be omitted from the triangulation, saving time (but giving in an incomplete
triangulation).
if (opt & 2 && i < npts && j < npts && k < npts) return;
d0 = storetriangle(r,i,j);           Create three triangles and queue them
tasks[++ntask] = r; taski[ntask] = i; taskj[ntask] = j;       for legal edge tests.
d1 = storetriangle(r,j,k);
tasks[++ntask] = r; taski[ntask] = j; taskj[ntask] = k;
d2 = storetriangle(r,k,i);
tasks[++ntask] = r; taski[ntask] = k; taskj[ntask] = i;
erasetriangle(i,j,k,d0,d1,d2);       Erase the old triangle.
while (ntask) {                       Legalize edges recursively.
    s=taski[ntask]; i=taski[ntask]; j=taskj[ntask--];
    key = hashfn.int64(j) - hashfn.int64(i);           Look up fourth point.
    if ( ! linehash->get(key,l) ) continue;           Case of no triangle on other side.
    if (incircle(pts[l],pts[j],pts[s],pts[i]) > 0.0){ Needs legalizing?
        d0 = storetriangle(s,l,j);                   Create two new triangles
        d1 = storetriangle(s,i,l);
        erasetriangle(s,i,j,d0,d1,-1);               and erase old ones.
        erasetriangle(l,j,i,d0,d1,-1);
        key = hashfn.int64(i)-hashfn.int64(j);       Erase line in both directions.
        linehash->erase(key);
        key = 0 - key;                               Unsigned, hence binary minus.
        linehash->erase(key);
        Two new edges now need checking:
        tasks[++ntask] = s; taski[ntask] = l; taskj[ntask] = j;
        tasks[++ntask] = s; taski[ntask] = i; taskj[ntask] = l;
    }
}
}
}

```

The only pieces left are the utility functions for finding the triangle that contains a point, and for storing and erasing a triangle. When we “erase” a triangle, we actually only mark it as inactive in the current triangulation, and we set its daughters in the descendency tree, as already discussed.

```

Int Delaunay::whichcontainspt(const Point<2> &p, Int strict) {           delaunay.h
    Given point p, return index in thelist of the triangle in the triangulation that contains it, or
    return -1 for failure. If strict is nonzero, require strict containment, otherwise allow the point
    to lie on an edge.
    Int i,j,k=0;
    while (thelist[k].stat <= 0) {           Descend in tree until reach a “live” triangle.
        for (i=0; i<3; i++) {               Check up to three daughters.
            if ((j = thelist[k].d[i]) < 0) continue;           Daughter doesn't exist.
            if (strict) {
                if (thelist[j].contains(p) > 0) break;
            } else {
                if (thelist[j].contains(p) >= 0) break;           Yes, descend on this branch.
            }
        }
        if (i == 3) return -1;               No daughters contain the point.
        k = j;                               Set new mother.
    }
    return k;                               Normal return.
}

```

```

void Delaunay::erasetriangle(Int a, Int b, Int c, Int d0, Int d1, Int d2) {
Erase triangle abc in trihash and inactivate it in thelist after setting its daughters.
    Ullong key;
    Int j;
    key = hashfn.int64(a) ^ hashfn.int64(b) ^ hashfn.int64(c);
    if (trihash->get(key,j) == 0) throw("nonexistent triangle");
    trihash->erase(key);
    thelist[j].d[0] = d0; thelist[j].d[1] = d1; thelist[j].d[2] = d2;
    thelist[j].stat = 0;
    ntri--;
}

Int Delaunay::storetriangle(Int a, Int b, Int c) {
Store a triangle with vertices a, b, c in trihash. Store its points in linehash under keys to
opposite sides. Add it to thelist, returning its index there.
    Ullong key;
    thelist[ntree].setme(a,b,c,&pts[0]);
    key = hashfn.int64(a) ^ hashfn.int64(b) ^ hashfn.int64(c);
    trihash->set(key,ntree);
    key = hashfn.int64(b)-hashfn.int64(c);
    linehash->set(key,a);
    key = hashfn.int64(c)-hashfn.int64(a);
    linehash->set(key,b);
    key = hashfn.int64(a)-hashfn.int64(b);
    linehash->set(key,c);
    if (++ntree == ntreamax) throw("thelist is sized too small");
    ntri++;
    return (ntree-1);
}

```

You might wonder how to get the answer *out* of our Delaunay structure. We have not provided a function for this, because it so much depends on what you want to do with the answer. The general idea, however, is that you just loop through `thelist[j]` for $0 \leq j < nlist$. Each element is a `Triel`. If its value of `save` is ≤ 0 , ignore it and go on. If it is 1, then the element represents a triangle in the final Delaunay triangulation. There should be `ntri` of these elements in all. The element's array `p[]` has integers that point to the triangle's three points in your vector of points. Several routines in the next section mine the Delaunay structure for points, edges, or triangles and can be used as template examples.

Figure 21.6.6 shows sample output for a Delaunay triangulation of 300 points.

CITED REFERENCES AND FURTHER READING:

- Guibas, L.J., Knuth, D.E., and Sharir, M. 1992, "Randomized Incremental Construction of Delaunay and Voronoi Diagrams," *Algorithmica*, vol. 7, pp. 381–413.[1]
- Lischinski, D. 1994, "Incremental Delaunay Triangulation," in *Graphics Gems IV*, Heckbert, P.S., ed. (Cambridge, MA: Academic Press). [Shows use of linked data structure instead of our use of hash memory.]
- de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 9.[2]
- O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press), §5.3.